Lecture 4: Equilibrium — Aiyagari

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 14, 2026

University of Toronto

Stages and Outer Loops

Stages

- Heterogeneous-agent macro models are made by combining **Stages** with **Outer Loops**.
- **Stages** are the basic building blocks. They define household problems, which are the **hard part**.
- Because **Stages** are conceptually **well-defined**, **modular**, and **composable**, complex macro models can be built by simply combining **Stages**.

Stages Solve the Household Block

- A **Stage** defines how households make decisions given their **environment**: **prices** and **parameters**.
- **Stages** compose into the **household block**: value functions and populations given prices and parameters.
- That gives us **aggregates**: goods demand, capital demand, labour supply, **as a function of** prices, parameters, market tightness, etc.
- In a sense, you can think of the household block as representing **one side** of the economy, sort of like a big aggregate demand curve.

Why Stages?

- Stages are **composable**: complex models built by combining Stages.
- Stages have **precise mathematical definitions**: a model can be defined and solved without worrying about implementation details.
- Stages are **standardized**: optimized implementations exist that can be downloaded and reused (e.g. `HouseholdStages.jl`, the SSJ Toolkit, or ECON-ARK).

How the Household Block Fits In

The **household block** (defined and solved using **Stages**) is one part of a model.

You can think of the rest of a model as consisting of:

- A system of equations describing the rest of the economy—firms, government, etc.—often implemented as a **directed acyclic graph** (DAG) of functions.
- A series of **Outer Loops** that solve for equilibrium prices, steady states, transition paths, or global solutions with aggregate uncertainty.

Outer Loops: Examples

Equilibrium: Search over prices against **market clearing** or **excess demand** objective.

Steady State: Search over population distributions Λ against **population-evolution** $\Lambda^{\text{end}} - \Lambda^{\text{start}'}$ objective.

Calibration: Search over parameters against **model fit** $\text{moment}^{\text{model}} - \text{moment}^{\text{data}}$ objective.

Perfect Foresight Transition: Search over **paths** of market-clearing prices against path-of-excess-demand objective.

Aggregate Uncertainty: Search over **global value functions** against **outer Bellman objective**.

Aiyagari Model

The Aiyagari model has these three blocks:

1. A **household block** made of **Stages**.
2. A **firm** that turns capital and labour into goods.
3. A notion of **equilibrium** (steady state).

Familiar economics:

1. A demand side.
2. A supply side.
3. A notion of equilibrium.

Aiyagari Household Block

Same As Last Time

The three-Stage household block from last time::

1. Income shock (Markov on z).
2. Income (cash-on-hand $b = R\,b^{\text{end}} + z$).
3. Consumption–saving (grid search on b^{end}).

Steady-state solution given prices:

- VFI: Value function V and policy $c^\star$.
- Forward simulation: Stationary distribution $\lambda^\star$.
- Today: Make R an endogenous *price* we have to solve for.

Solve the household block at a given R

```
function solve_aiyagari_steady_state(env; verbosity=1)
    V_end = solve_vfi(env; verbosity)
    return (; V_end, λ = find_steady_state_population(V_end, env; verbosity))
end
```

- env contains prices and parameters.
- In one function call, solve for steady state.
- Can think of as, among other things, one big function $R \mapsto \bar{K}$.

Aggregates

Aggregate Moments as Inner Products

Every aggregate moment is a **dual vector**: an inner product with $\lambda^\star$.

$$\bar{K} = \sum_{i,j} b_i \, \lambda^\star_{ij}$$

$$\bar{L} = \sum_{i,j} \lambda^\star_{ij}$$

$$\bar{C} = \sum_{i,j} c^\star_{ij} \, \lambda^\star_{ij}$$

$$\bar{V} = \sum_{i,j} V_{ij} \, \lambda^\star_{ij}$$

compute_household_aggregates

```
function compute_household_aggregates(;param_vals...)
    params = get_params(;param_vals...)
    (V_end, λ) = solve_aiyagari_steady_state(params)

    b_grid_policy = policy_function(V_end, params)
    c_policy = get_c_policy(params.b_grid, b_grid_policy)

    L_agg = sum( λ)
    K_agg = sum(params.b_grid .* λ)
    V_agg = sum(V_end .* λ)
    C_agg = sum(c_policy .* λ)

    return (; V_end, λ, L_agg, K_agg, V_agg, C_agg)
end
```

The Firm

Cobb–Douglas

Representative firm with Cobb–Douglas production:

$$Y = K^{\alpha} L^{1-\alpha}.$$

This is the supply side of the model.

In code,

```
compute_Y(K, L; α = 1/3) = K^ α * L^ (1 - α)
```

Equilibrium

Goods market clearing

In steady state,

$$\bar{C} = Y.$$

Define the *excess demand* for goods:

$$\text{excess\_demand}(R) = \frac{\bar{C}(R) - Y(R)}{Y(R)}.$$

Equilibrium R is the **root** of this function: The value solving $\text{excess\_demand}(R) = 0$.

compute_excess_demand

We can code up this function,

```
function compute_excess_demand(R)
    res = compute_household_aggregates(;R)
    Y_agg = compute_Y(res.K_agg, res.L_agg)
    return excess_demand = (res.C_agg - Y_agg)/Y_agg
end
```

The chain of direct effects is:

$$R \to V \to c^\star \to \lambda^\star \to (\bar{K}, \bar{C}) \to Y.$$

Solving for equilibrium R

```
tol = 0.01
lr = 0.001

R = 1.01
err = Inf
while err > tol
    excess_demand = compute_excess_demand(R)

    R_new = R - excess_demand*lr
    err = abs(excess_demand)

    R = R_new

    println("R=$R\t excess_demand=$excess_demand")
end
```

- Tatonnement: A simple gradient-style update: update R in the direction of excess goods demand